MLOps: Machine Learning Operations

Designing Production ML Systems

Google MLOps Whitepaper | CD4ML | Sculley et al. | MLflow | W&B;

What is MLOps?

MLOps (Machine Learning Operations) extends DevOps practices to machine learning systems. It addresses the unique challenges of ML: data versioning, experiment tracking, model training, evaluation, deployment, monitoring, and retraining. The goal is reproducible, reliable, scalable ML in production.

1. MLOps Maturity Model

Level	Name	Characteristics
0	Manual Process	Jupyter notebooks, manual training, no versioning, one-time deployment
1	ML Pipeline Automation	Automated training, CT pipeline, experiment tracking, basic monitoring
2	CI/CD Pipeline	Full automation, automated testing, model registry, continuous training & delivery
3	MLOps Platform	Feature stores, unified platform, drift detection, A/B testing, auto-retraining

2. Core MLOps Components

Data Versioning (DVC, LakeFS): Version datasets and ML artifacts like code. DVC stores metadata in git and data in S3/GCS. Enables reproducibility: git checkout v1.2 → get exact training data.

Experiment Tracking (MLflow, W&B;, Neptune): Log hyperparameters, metrics, artifacts per run. Compare runs visually. W&B; is the industry standard for deep learning; MLflow for MLOps pipelines.

Model Registry (MLflow, W&B;, SageMaker): Centralized store of model versions with metadata (accuracy, training data, lineage). Lifecycle stages: Staging → Production → Archived.

Feature Store (Feast, Tecton, Hopsworks): Centralized repository of features with point-in-time correctness. Prevents train-serve skew. Online store (Redis) + offline store (BigQuery/Snowflake).

CI/CD for ML (GitHub Actions, Kubeflow, Vertex AI): Automated pipelines trigger on: new data, code change, schedule, or performance drift. Tests: data validation, model evaluation, bias checks,

latency benchmarks.

Model Monitoring (Evidently, Arize, WhyLabs): Track data drift (input distribution change), concept drift (prediction distribution change), and performance metrics in production. Alert and retrain on threshold breach.

3. Model Serving Patterns

REST API (FastAPI + Docker): Simple, flexible. Single-model serving. 50–500ms latency.

Batch inference: Process large datasets offline. Use Spark, Ray Data, or Airflow DAGs.

BentoML: Framework-agnostic model serving with built-in batching and monitoring.

Triton Inference Server (NVIDIA): Multi-framework, multi-GPU, dynamic batching. Optimized for throughput.

vLLM: PagedAttention for LLM serving. 10–24x throughput vs HuggingFace. Standard for LLM inference.

TorchServe / TensorRT: PyTorch/ONNX model optimization and serving for production.

4. MLOps Stack Example (2024)

```
# Modern MLOps Stack for LLM Applications

Data Layer:
  Storage:      S3 / GCS / Azure Blob
  Versioning:   DVC or LakeFS
  Processing:   Apache Spark / dbt / Pandas

Training Layer:
  Framework:    PyTorch + HuggingFace Transformers
  Tracking:     Weights & Biases
  Compute:      AWS SageMaker / GCP Vertex AI / RunPod
  Distributed:  DeepSpeed ZeRO / FSDP

Serving Layer:
  LLM Inference: vLLM with OpenAI-compatible API
  Gateway:       Kong / AWS API Gateway
  Cache:         Redis (semantic caching with GPTCache)

Monitoring:
  Metrics:      Prometheus + Grafana
  Drift:        Evidently AI
  Logging:      ELK Stack / Datadog
  Tracing:      LangSmith / LangFuse (for LLM traces)

Orchestration:
  Pipelines:    Apache Airflow / Kubeflow Pipelines
  CI/CD:        GitHub Actions + ArgoCD
```

```
Container:    Docker + Kubernetes (EKS/GKE)
```